



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

ОТЧЁТ ПО ПРАКТИКЕ В ПРОФИЛЬНОЙ СФЕРЕ
**Тема практики: «PostgreSQL в Docker. Создание контейнера и его
запуск с базой данных»**

Отчет представлен к
рассмотрению:

Студент группы ИКБО-50-23

«31» мая 2025 г.

Астахова С.П.

(подпись)

Отчет утвержден.

Допущен к защите:

Руководитель практики

«31» мая 2025 г.

Тарланов А.Т.

(подпись)

Москва 2025 г.



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

**ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ НА ПРАКТИКУ В ПРОФИЛЬНОЙ
СФЕРЕ**

**Студент 2 курса учебной группы ИКБО-50-23
Астахов Сергей Павлович**

Время практики: с 26 мая 2025 г. по 31 мая 2025 г.

Должность на практике: студент

1. ЦЕЛЕВАЯ УСТАНОВКА: PostgreSQL в Docker. Создание контейнера и его запуск с базой данных

2. СОДЕРЖАНИЕ ПРАКТИКИ:

2.1 Введение

2.2 Основная работа

2.3 Заключение

3. ОРГАНИЗАЦИОННО-МЕТОДИЧЕСКИЕ УКАЗАНИЯ: в процессе практики рекомендуется использовать периодические издания и отраслевую литературу годом издания не старше 10 лет.

СОГЛАСОВАНО:

Руководитель практики
«26» мая 2025 г.

(подпись) (Тарланов А.Т.)

Задание получил
«26» мая 2025 г.

(подпись) (Астахов С.П.)



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

**РАБОЧИЙ ГРАФИК ПРОВЕДЕНИЯ
практики в профильной сфере**

студента Астахова С.П. (Родит. Падеж) 2 курса группы ИКБО-50-23.

№	Сроки выполнения	Этап	Отметка о выполнении
1	26.05.2025	Подготовительный этап, включающий в себя организационное собрание (Вводная лекция о порядке организации и прохождения практики)	выполнено
2	с 26.05.2025 по 29.05.2025	Выполнение задания по практике в соответствии с выданным заданием студента. (Мероприятия по сбору, обработке и структурированию материала, выполнение поставленной задачи)	выполнено
3	с 29.05.2025 по 30.05.2025	Подготовка отчета по практике (Оформление материалов отчета в полном соответствии с требованиями на оформление учебных работ студентов)	выполнено
4	31.05.2025	Представление отчета по практике к защите посредством загрузки на портал ДПО (https://online-dpo.mirea.ru/)	выполнено

Согласовано:

Руководитель практики _____/Тарланов А.Т./

Обучающийся _____/ Астахов С.П./

Содержание

<i>Тема практики: «PostgreSQL в Docker. Создание контейнера и его запуск с базой данных»</i>	<i>1</i>
<i>РАБОЧИЙ ГРАФИК ПРОВЕДЕНИЯ</i>	<i>3</i>
<i>1. ВВЕДЕНИЕ.....</i>	<i>5</i>
<i>1.1 PostgreSQL</i>	<i>5</i>
<i>1.2 Docker</i>	<i>11</i>
<i>2. ОСНОВНАЯ РАБОТА.....</i>	<i>18</i>
<i>3. ЗАКЛЮЧЕНИЕ</i>	<i>24</i>
<i>4. СПИСОК ЛИТЕРАТУРЫ</i>	<i>26</i>

1. ВВЕДЕНИЕ

Перед написанием работы, стоит вообще разобраться с чем мы будем иметь дело, как все работает и какие утилиты мы будем использовать. Опишем это подразделах.

1.1 PostgreSQL

PostgreSQL – это платформа свободной объектно-реляционной системы управления для создания реляционной базы данных. С помощью этого мощного инструмента можно структурировать множество данных в таблицы и позже обращаться к элементам по индексам. PostgreSQL базируется на языке SQL и поддерживает многие из возможностей стандарта SQL:2011.

Благодаря базам данных, разработчики могут систематизировать большое количество данных в одну структура и позже, при создании приложения, легко обращаться к этим данным.

Следует отметить, что PostgreSQL поддерживает концепции, присущие как реляционным базам данных, так и объекта-ориентированным языка программирования, что ясно из названия, но также еще работает и с нереляционными базами данных. Уточним об отличиях этих двух категориях поточнее:

- **Реляционные базы данных -**

Это тип базы данных, основанный на реляционной модели, которая организует данные в виде таблиц. Каждая строка в таблице представляет запись, а каждый столбец – атрибут данных. Изначально, при создании вообще базы данных, реляционной структуры не было. После ее сделали на основе такой дисциплины как «реляционная математика», что послужила толчком к новым возможностям. Основная структура и компоненты реляционной базы данных:

- *Таблицы(отношения)* - двумерные структуры для хранения данных. Каждая таблица имеет уникальное имя и представляет определённый тип сущности.
- *Строки (кортежи)* - горизонтальные наборы значений, представляющие экземпляры сущностей.
- *Столбцы (атрибуты)* - вертикальные наборы значений определённого типа.
- *Ключи* - специальные атрибуты для идентификации и связывания данных. Выделяют первичный ключ (уникальный идентификатор строки) и внешние ключи (для создания связей между таблицами).
- *Схема базы данных* - формальное описание структуры таблиц, их атрибутов, типов данных и взаимосвязей.

Пример представлен на рис. 1:



Рисунок 1 – Реляционная база данных

- **Нереляционные базы данных (NoSQL) –**

это системы хранения информации, в которых не используется табличная схема строк и столбцов. Вместо этого применяется модель хранения, оптимизированная под конкретный тип хранимых данных. NoSQL появились первыми базами данных в 80-х годах прошлого века. С течением того, как популярность их упала после создания реляционных баз данных, NoSQL понемногу замещали из-за ненадобности, и, относительно недавно, мы снова стали использовать нереляционные базы данных, как в примере для создания кэша или для других отдельных случаях. Разберем типы НБЗ:

- «Ключ-значение». Данные хранятся в парах, где ключ выступает уникальным идентификатором.
- *Документно-ориентированные*. Информация записывается в специальный документ, обычно в формате JSON.
- *Графовые*. Данные сохраняются в виде сложно связанных графов, что упрощает их хранение, навигацию и поиск.
- *Столбчатые (колоночные)*. Хранят данные в столбцах, что позволяет быстро работать с большими объемами информации.

Пример представлен на рис.2:

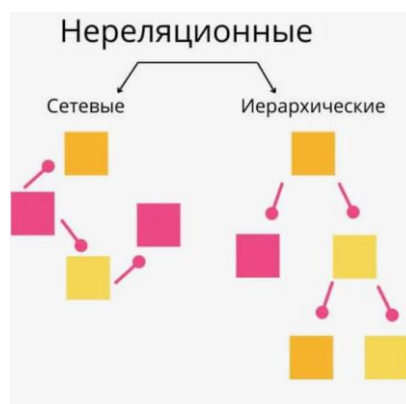


Рисунок 2 – Нереляционная базы данных

В каждом из систем хранения данных есть как преимущества, так и недостатки, но мы не будем тут этого описывать, но поговорим еще об преимуществах и недостатках PostgreSQL и почему именно это программный продукт мы взяли как основой производственной практики.

Преимущества PostgreSQL:

- **Открытый исходный код.** Позволяет свободно использовать, модифицировать и распространять проект.
- **Поддержка сложных типов данных.** Включает JSON, XML и пользовательские типы данных, что позволяет хранить и обрабатывать не только структурированные данные, но и более сложные, например документы и мультимедиа.
- **Высокая производительность.** Поддерживает параллельную обработку запросов, индексы и механизмы, которые ускоряют поиск и манипуляцию данными.
- **Масштабируемость.** Поддерживает вертикальное и горизонтальное масштабирование, что позволяет эффективно работать как с малыми, так и с очень большими объёмами данных.
- **Высокий уровень безопасности.** Предоставляет механизмы для защиты данных, такие как шифрование, аутентификация и авторизация, а также поддерживает строгие права доступа.
- **Кроссплатформенность.** Поддерживает работу со всеми распространёнными операционными системами: Linux, Windows, BSD, macOS и Solaris.

Недостатки PostgreSQL:

- **Сложность настройки.** Разнообразие конфигураций может создавать сложности для начинающих пользователей.
- **Высокое потребление ресурсов.** PostgreSQL может потреблять больше памяти и процессорного времени по сравнению с некоторыми другими СУБД, особенно при работе с большими объёмами данных и сложными запросами.
- **Ограниченное количество функций.** В сравнении с некоторыми коммерческими СУБД PostgreSQL может немного отставать в функциональности.
- **Сложности с резервным копированием и восстановлением.** Особенно для больших массивов данных требуется тщательное планирование и реализация, чтобы обеспечить надёжность резервного копирования и возможность быстрого восстановления в случае сбоя.

На основании этих факторов, был сделан выбор, взять за основу PostgreSQL.

1.2 Docker

Docker — это мощная платформа с открытым исходным кодом, предназначенная для создания, развертывания и управления контейнеризированными приложениями. Она позволяет разработчикам упаковывать приложения вместе со всеми зависимостями и настройками в изолированные контейнеры, которые можно запускать в любой среде — будь то Linux, Windows или macOS.

Благодаря Docker процесс разработки, тестирования и развертывания становится быстрее и надежнее. Разработчики больше не сталкиваются с

проблемами совместимости, поскольку контейнеры работают одинаково на любой системе, поддерживающей Docker.

Особенности контейнеров:

1. Изолированность

- Каждый контейнер работает в собственной среде, не влияя на другие процессы.

- Позволяет избежать конфликтов версий библиотек и зависимостей.

2. Переносимость

- Контейнеры можно легко переносить между различными системами (локальная машина, облако, сервер).

- Обеспечивают идентичное поведение приложения в разных окружениях.

3. Легковесность

- В отличие от виртуальных машин, контейнеры используют общее ядро ОС, что снижает нагрузку на систему.

- Быстрый запуск и минимальные накладные расходы.

4. Масштабируемость

- Docker позволяет легко масштабировать приложения, запуская несколько экземпляров контейнеров.

- Интеграция с оркестраторами (Kubernetes, Docker Swarm) для управления кластерами.

5. Версионность и воспроизводимость

- Образы Docker можно версионировать, что упрощает откат изменений.

- Обеспечивает идентичную среду для всех участников команды.

Основные компоненты Docker:

1. Docker Engine

- Основной движок, отвечающий за создание и управление контейнерами.

Включает в себя:

- Docker Daemon (фоновый процесс, управляющий контейнерами).
- Docker CLI (интерфейс командной строки для взаимодействия с демоном).

2. Docker Images (Образы)

- Шаблоны для создания контейнеров.
- Содержат ОС, зависимости и код приложения.

3. Docker Containers (Контейнеры)

- Запущенные экземпляры образов.
- Изолированные среды выполнения приложений.

4. Docker Registry (Реестр)

- Хранилище образов (например, Docker Hub — публичный реестр).
- Позволяет скачивать и загружать образы.

5. Docker Compose

- Инструмент для определения и управления мультиконтейнерными приложениями через YAML-файлы.

6. Dockerfile

- Текстовый файл с инструкциями для сборки образа.

Как устроен образ Docker:

Образ Docker — это ****многослойная (layered) файловая система****, где каждый слой соответствует определенной инструкции в Dockerfile.

- Базовый слой (Base Image)
- Обычно содержит минимальную ОС (например, `alpine`, `ubuntu`).
- Дополнительные слои
- Каждая команда в Dockerfile (`RUN`, `COPY`, `ADD` и др.) создает новый слой.
- Итоговый образ
 - Объединяет все слои в единую файловую систему (Read-Only).
 - При запуске контейнера добавляется временный слой (Read-Write).

Преимущества многослойности:

- Экономия места (общие слои между образами).
- Ускорение сборки (кеширование слоев).
- Возможность отката к предыдущим версиям.

Пример того как устроен образ в Docker представлен на рис. 3:

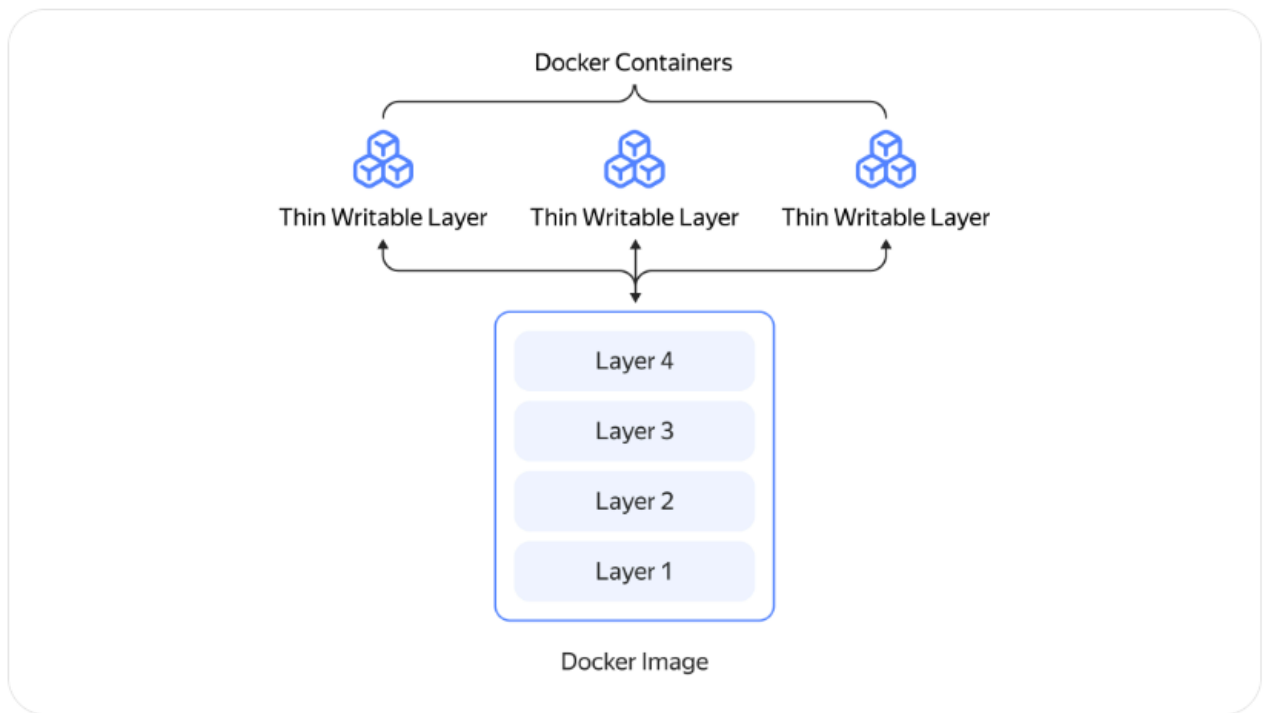


Рисунок 3 – Как устроен образ Docker

Создание образа Docker:

1. Написание Dockerfile

Пример простого Dockerfile для Python-приложения:

```
dockerfile

# Базовый образ
FROM python:3.9-slim

# Рабочая директория
WORKDIR /app

# Копирование зависимостей
COPY requirements.txt

RUN pip install -r requirements.txt

# Копирование кода приложения
```

```
COPY
```

```
# Команда для запуска
```

```
CMD ["python", "app.py"]
```

2. Сборка образа

```
docker build -t my-python-app .
```

- `-t` — задает имя и тег образа.
- `.` — путь к Dockerfile.

3. Запуск контейнера

```
docker run -d -p 5000:5000 my-python-app
```

- `-d` — запуск в фоновом режиме.
- `-p` — проброс портов (хост:контейнер).

4. Публикация образа

```
docker login
```

```
docker tag my-python-app username/my-python-app:v1
```

```
docker push username/my-python-app:v1
```

2. ОСНОВНАЯ РАБОТА

Сначала скачаем только клиентские библиотеки, потому что в docker есть уже встроенный сервер postgresSQL, которым мы и воспользуемся позже.

Дальше, всю проделанную работу я буду делать на archlinux. Установим PostgreSQL клиент на операционную систему:

```
sergey@archlinux ~ [1]> sudo pacman -S postgresql-libs
предупреждение: postgresql-libs-17.4-4 не устарел -- переустанавливается
разрешение зависимостей...
проверка конфликтов...

Пакет (1)                Старая версия  Новая версия  Изменение размера
extra/postgresql-libs  17.4-4        17.4-4        0,00 MiB

Будет установлено:  8,82 MiB
Изменение размера:  0,00 MiB

:: Приступить к установке? [Y/n] Y
(1/1) проверка ключей [-----] 100%
(1/1) проверка целостности пакета [-----] 100%
(1/1) загрузка файлов пакетов [-----] 100%
(1/1) проверка конфликтов файлов [-----] 100%
(1/1) проверка доступного места [-----] 100%
:: Обработка изменений пакета...
(1/1) переустановка postgresql-libs [-----] 100%
:: Запуск post-transaction hooks...
(1/1) Arming ConditionNeedsUpdate...
sergey@archlinux ~-> |
```

Рисунок 1 — Установка postgresql-libs

Создадим папку, где проделаем всю работу и там же создадим sql скрипт:

```
sergey@archlinux ~-> cd Desktop/
sergey@archlinux ~/Desktop> mkdir ./postgres-docker
sergey@archlinux ~/Desktop> micro init.sql
```

Рисунок 2 — Создание папки для задания и sql файла

В файле создадим таблицу и в ней заполним две строчки таблицы:

```
1 CREATE TABLE IF NOT EXISTS users (
2     id SERIAL PRIMARY KEY,
3     username VARCHAR(50) NOT NULL,
4     email VARCHAR(100) UNIQUE NOT NULL,
5     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
6 );
7
8 INSERT INTO users (username, email) VALUES
9     ('arch_user', 'arch@example.com'),
10    ('docker_fan', 'docker@example.com');
11
```

Рисунок 3 — Создание таблицы и заполнение ее значениями

Скачаем

```
sergey@archlinux ~/Desktop> sudo pacman -S docker docker-compose
[sudo] пароль для sergey:
разрешение зависимостей...
проверка конфликтов...

Пакет (4)          Новая версия  Изменение размера  Размер загрузки
extra/containerd    2.1.1-1      87,32 MiB          19,33 MiB
extra/runc          1.3.0-1      10,29 MiB           3,34 MiB
extra/docker        1:28.1.1-1   94,85 MiB          23,67 MiB
extra/docker-compose 2.36.2-1     62,61 MiB          13,75 MiB

Будет загружено:    60,08 MiB
Будет установлено: 255,07 MiB

:: Приступить к установке? [Y/n] Y
:: Получение пакетов...
runc-1.3.0-1-x86_64      3,3 MiB  1002 KiB/s  00:03 [-----] 100%
containerd-2.1.1-1-x86_64 19,3 MiB   2,57 MiB/s  00:08 [-----] 100%
docker-compose-2.36.2-1-x86_64 13,7 MiB  1671 KiB/s  00:08 [-----] 100%
docker-1:28.1.1-1-x86_64 23,7 MiB   2,13 MiB/s  00:11 [-----] 100%
Total (4/4)             60,1 MiB   5,39 MiB/s  00:11 [-----] 100%
(4/4) проверка ключей [-----] 100%
(4/4) проверка целостности пакета [-----] 100%
(4/4) загрузка файлов пакетов [-----] 100%
(4/4) проверка конфликтов файлов [-----] 100%
(4/4) проверка доступного места [-----] 100%
:: Обработка изменений пакета...
(1/4) установка runc [-----] 100%
Дополнительные зависимости для 'runc'
criu: checkpoint support
(2/4) установка containerd [-----] 100%
(3/4) установка docker [-----] 100%
Дополнительные зависимости для 'docker'
btrfs-progs: btrfs backend support
```

Рисунок 4 — Скачивание docker

Включим его прямо сейчас:

```
sergey@archlinux ~/Desktop> sudo systemctl enable --now docker
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' → '/usr/lib/systemd/system/docker.service'.
sergey@archlinux ~/Desktop> |
```

Рисунок 5 — Включаем docker

Создадим файл dockerfile и заполним его.

```
1 Dockerfile
2 FROM postgres:15
3
4 # Копируем скрипт инициализации
5 COPY init.sql /docker-entrypoint-initdb.d/
6
7 # Опционально: настраиваем переменные окружения по умолчанию
8 ENV POSTGRES_USER=archuser
9 ENV POSTGRES_PASSWORD=archpass
10 ENV POSTGRES_DB=archdb
11
12 # Открываем порт PostgreSQL
13 EXPOSE 5432
14
```

Рисунок 6 — Содержание docerfile

Для того чтоб docker работал нормально, нам получить права на управления docker:

```
sergey@archlinux ~/D/postgres-docker> sudo usermod -aG docker $USER
```

Рисунок 7 — Добавление пользователю прав

```
sergey@archlinux ~/D/postgres-docker> docker build -t my-postgres .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 3.072kB
Step 1/6 : FROM postgres:15
15: Pulling from library/postgres
61320b01ae5e: Pulling fs layer
9bc1585b18dd: Pulling fs layer
9a327a5b84b5: Pulling fs layer
b523a75b1854: Pulling fs layer
220845a5e2aa: Pulling fs layer
e12ef98d9f64: Pulling fs layer
61887648e4b4: Pulling fs layer
eeabd2f53a51: Pulling fs layer
9a7cf2dbf421: Pulling fs layer
82c0a6ef605e: Pulling fs layer
37a03f155251: Pulling fs layer
4afb31615866: Pulling fs layer
7a950735fd5d: Pulling fs layer
2e9810367290: Pulling fs layer
eeabd2f53a51: Waiting
9a7cf2dbf421: Waiting
82c0a6ef605e: Waiting
37a03f155251: Waiting
4afb31615866: Waiting
7a950735fd5d: Waiting
2e9810367290: Waiting
b523a75b1854: Waiting
220845a5e2aa: Waiting
e12ef98d9f64: Waiting
61887648e4b4: Waiting
9bc1585b18dd: Verifying Checksum
9bc1585b18dd: Download complete
```

Рисунок 8 — Запуск билда(часть 1)

Соберем образ:

```

37a03f155251: Pull complete
4afb31615866: Pull complete
7a950735fd5d: Pull complete
2e9810367290: Pull complete
Digest: sha256:f57a3bdbf044f0b213fdc99f35a0d21c401608bf41f063176ec00c51df9655f7
Status: Downloaded newer image for postgres:15
----> 1508ba0c35e0
Step 2/6 : COPY init.sql /docker-entrypoint-initdb.d/
----> 667612c155de
Step 3/6 : ENV POSTGRES_USER=archuser
----> Running in dd008841c0e2
----> Removed intermediate container dd008841c0e2
----> ea9aad1ec8c0
Step 4/6 : ENV POSTGRES_PASSWORD=archpass
----> Running in 808127b17ce8
----> Removed intermediate container 808127b17ce8
----> d4fb1b497509
Step 5/6 : ENV POSTGRES_DB=archdb
----> Running in fee3934f358c
----> Removed intermediate container fee3934f358c
----> 81e37446f219
Step 6/6 : EXPOSE 5432
----> Running in e53e36fdf281
----> Removed intermediate container e53e36fdf281
----> d76a9b825ac5
Successfully built d76a9b825ac5
Successfully tagged my-postgres:latest
sergey@archlinux ~/D/postgres-docker> |

```

Рисунок 9 - Запуск билда(часть 2)

```

sergey@archlinux ~/D/postgres-docker> docker run -d \
--name my-postgres-container \
-p 5432:5432 \
-v pg_data:/var/lib/postgresql/data \
my-postgres
3001fb53d1f1c1c1b863a167d2e4c04f8136b7157197081ebc0c03065d93e131

```

Рисунок 10 — Запуск контейнера

Проверим работу:

```

sergey@archlinux ~/D/postgres-docker> docker exec -it my-postgres-container psql -U archuser -d archdb -c "SELECT * FROM users;"

```

id	username	email	created_at
1	arch_user	arch@example.com	2025-05-31 05:04:43.989976
2	docker_fan	docker@example.com	2025-05-31 05:04:43.989976

(2 rows)

Рисунок 11 — Вывод таблицы

Как работает наша система:

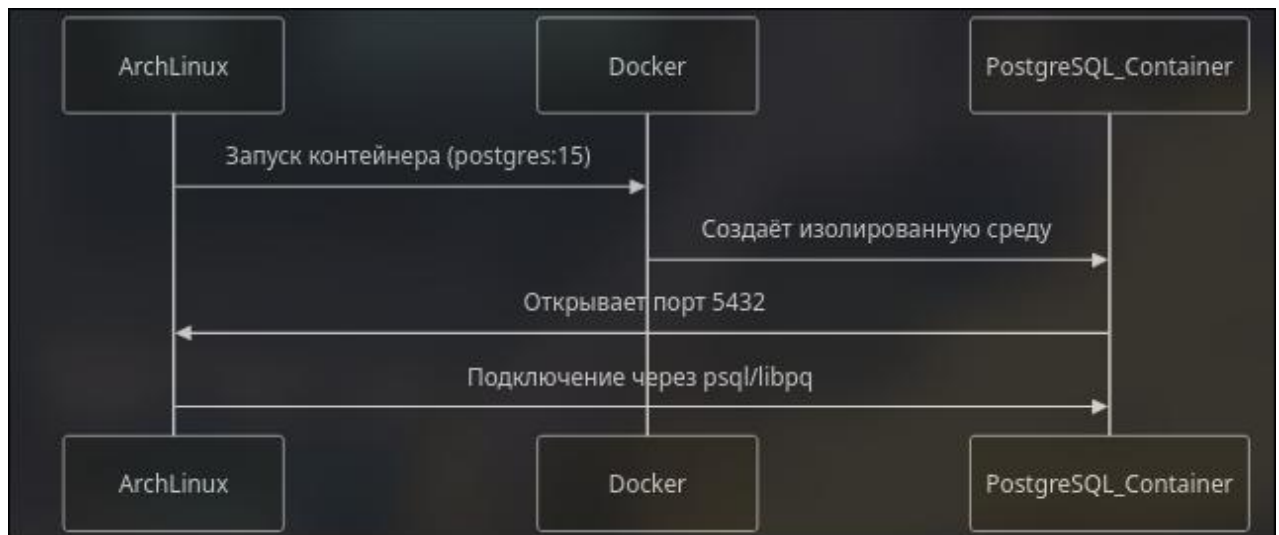


Рисунок 12 — Как работает программа

3. ЗАКЛЮЧЕНИЕ

В рамках производственной практики была разработана база данных на основе PostgreSQL с использованием Docker-контейнеризации. Этот подход позволил создать изолированное и переносимое окружение, где все компоненты СУБД — от сервера до конфигурации и данных — работали в едином контейнере, не затрагивая основную систему. Особое внимание было уделено адаптации решения под Arch Linux: мы использовали минимальные клиентские библиотеки (postgresql-libs) для подключения к контейнеру, избегая установки полноценного сервера в систему. Это обеспечило чистоту окружения и простоту управления, а автоматизация через Docker Compose и systemd unit-файлы позволила легко разворачивать и масштабировать базу данных.

Применение Docker не только ускорило процесс разработки, но и решило ключевые проблемы согласованности сред между участниками команды. Контейнеризация дала возможность быстро поднимать и останавливать экземпляры БД, сохраняя данные между запусками через volumes, а также интегрировать PostgreSQL с другими сервисами в единой Docker-сети. В результате мы получили отказоустойчивое решение, готовое к использованию как в разработке, так и в production-среде, с перспективами дальнейшего расширения функциональности за счёт репликации и специализированных расширений.

4. СПИСОК ЛИТЕРАТУРЫ

1. Мамонтов К.Ю., Новиков А.А. Docker и контейнеризация: практика DevOps. – М.: БХВ-Петербург, 2021. – 288 с.
2. Docker Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/> – Дата обращения: 18.05.2025.
3. Официальный сайт PostgreSQL (postgresql.org). Содержит официальную документацию, каталог программного обеспечения и другие материалы.
4. Шкарупа А.В. Технологии DevOps и контейнеризация приложений. – СПб.: Питер, 2020. – 352 с.
5. Лекционный и практический материал Котилевец Игорь Денисович на сайте ДПО РТУ МИРЭА по курсу: Технологии Девопс.